

FAQ Chatbot Using NLP (Python)

This project demonstrates how to build a simple FAQ chatbot using Python and NLP techniques. The chatbot stores FAQs, preprocesses user text, compares similarity using TF-IDF and cosine similarity, and returns the most relevant answer.

Project Structure

```
faq_chatbot/  
■  
■■■ faq_chatbot.py  
■■■ faqs.json  
■■■ requirements.txt
```

Install Required Libraries

```
pip install nltk scikit-learn  
pip install spacy  
python -m spacy download en_core_web_sm
```

Sample FAQ Dataset (faqs.json)

```
[  
  {  
    "question": "What is your return policy?",  
    "answer": "You can return any product within 30 days."  
  },  
  {  
    "question": "How can I track my order?",  
    "answer": "Use the tracking link sent to your email."  
  }  
]
```

Main Features

- FAQ storage system
 - NLP preprocessing
 - TF-IDF vectorization
 - Cosine similarity matching
 - Chatbot response generation
 - Optional Streamlit web UI
-

Main Python Code (faq_chatbot.py)

```
import json  
import nltk  
import string  
  
from nltk.corpus import stopwords  
from nltk.tokenize import word_tokenize  
from sklearn.feature_extraction.text import TfidfVectorizer  
from sklearn.metrics.pairwise import cosine_similarity  
  
nltk.download('punkt')
```

```

nltk.download('stopwords')

with open("faqs.json", "r") as file:
    faqs = json.load(file)

questions = [faq["question"] for faq in faqs]
answers = [faq["answer"] for faq in faqs]

def preprocess(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))

    words = word_tokenize(text)

    stop_words = set(stopwords.words('english'))
    words = [word for word in words if word not in stop_words]

    return " ".join(words)

processed_questions = [preprocess(q) for q in questions]

vectorizer = TfidfVectorizer()
question_vectors = vectorizer.fit_transform(processed_questions)

def chatbot_response(user_input):
    processed_input = preprocess(user_input)

    input_vector = vectorizer.transform([processed_input])

    similarities = cosine_similarity(input_vector, question_vectors)

    best_match_index = similarities.argmax()
    best_score = similarities[0][best_match_index]

    if best_score < 0.3:
        return "Sorry, I couldn't understand your question."

    return answers[best_match_index]

print("FAQ Chatbot (type 'exit' to quit)")

while True:
    user_input = input("You: ")

    if user_input.lower() == "exit":
        print("Bot: Goodbye!")
        break

    response = chatbot_response(user_input)

    print("Bot:", response)

```

Example Output

```

FAQ Chatbot (type 'exit' to quit)

You: Can I return a product?
Bot: You can return any product within 30 days.

You: How do I pay?
Bot: We accept credit cards, PayPal, and UPI.

```

This project is suitable for beginners learning NLP and chatbot development. It demonstrates the practical use of text preprocessing, TF-IDF vectorization, and cosine similarity in building intelligent FAQ systems.